マインスイーパーで学ぶ

Bend 2 中級

標準ライブラリ・入出力・GUI・仕様と証明

プレビュー版

本書の目的は、入門編では十分に扱わなかった Bend 2 の機能を、型、引数、返り値、所有権、失敗時の挙動まで理解し、実際に使えるようになることです。マインスイーパーは学習を支える共通の題材です。本文で扱う一部の高度な機能は、GUI や外部環境が必要なため、動作条件を明示しています。

2026 年 10 月

はじめに

この本は『ライフゲームで学ぶはじめての Bend2』を読んだ人を対象とします。関数、パターンマッチ、再帰、アフィン性、do の基本は既習とします。難しいテクニックを身につけるよりも、**使える道具の一覧を把握し、必要な場面で正しい使い方を選べること**を重視します。

本書には、マインスイーパーとは直接関係のない独立したプログラムもあります。たとえばネットワーク通信や音声出力をゲームに無理に組み込むと、それらの本来の使用方法が分かりにくくなるためです。

Contents

はじめに i

I 標準ライブラリとデータ構造 1

1 Base を調べる 2

2 数値、文字、比較を正しく選ぶ 3

2.1 文字列の前処理と安全な変換 3

3 List と Array の詳しい使い方 5

3.1 List.filter と foldl を組み合わせる 5

3.2 Array.new の引数は長さではない 6

4 Map と Set を使う 7

4.1 Map.keys と Set の削除 8

5 失敗、選択肢、依存する値 10

II 入出力と並行処理 11

6 IO を読む 12

7 File とリソースの受け渡し 13

7.1 File.read による読み込みとラウンドトリップ 14

8 引数、時刻、乱数、プロセス 15

8.1 環境変数は常に存在するとは限らない 15

8.2 乱数の取得と失敗の伝播 16

8.3 外部プログラムを起動する 16

9 IO.fork、Chan とタイムアウト 17

9.1 独立した IO を開始して結果を待つ 17

10 TCP、UDP と外部環境 19

III Image・App・GUI 20

11 Image と四分木 21

12 App とイベントの処理 22

13 マインスイーパーの状態 23

13.1 Set を利用した 8 近傍の数え上げ 24

14 表示と入力をテストする 26

IV 仕様駆動開発と形式的証明 27

15 law を先に書く 28

16 等式、帰納法、補題と書き換え 30

17 盤面の不変条件 31

18 証明の範囲と信頼境界 32

V 並列実行とその他の応用 33

19 CPU、GPU、Array.atomic 34

20 C・JavaScript と外部連携 35

VI マインスイーパー統合と発展 36

21 一人で遊べるアプリにする 37

22 発展：運ゲー無しマインスイーパー 38

22.1 なぜ通常の盤面では推測が必要になるのか 38

22.2 最小の推論規則を Bend 2 で書く 38

22.3 複数の数字から推論する 40

22.4 公開情報と真の地雷配置を分離する 40

22.5 候補生成と検証を繰り返す 40

22.6 小盤面用の参考実装と検証 41

22.7 「運ゲー無し」の保証範囲 41

23 API 索引と次の学習 **42**

Part I

標準ライブラリとデータ構造

Chapter 1

Base を調べる

この章で学ぶこと：API の宣言、型、数量指定、結果を返す形を読み取る。

入門編では `import Base` を使いました。ここでは、`bend base` と `bend base Array` を使い、提供されている関数を自分で調べます。最新情報はhttps://github.com/bendlang/bend/blob/main/bend2/base.bendも参照してください。

例えば `Set.has` の戻り値は単なる `Bool` ではなく `Set() & Bool` です。「所属を調べる際に集合が消費され、再利用するには返された集合を受け取る」という設計を、この型だけから読み取れます。`Map.get` も元の辞書と値を組で返します。型名、消費する引数、戻ってくる所有物を最初に確認する習慣が重要です。

```
bend base
bend base Set
bend base Map
bend guide
```

すべての定義が通常のユーザー向け API とは限りません。`.go` や `.fin` など実装上のヘルパーも混在しています。最初は外側の公開操作を学び、実装を読む必要が生じたときに内部関数へ進みます。

演習　`Array.get`、`Set.has`、`File.open` を調べ、各関数について「入力」「結果」「失敗の有無」を記録してみましょう。

Chapter 2

数値、文字、比較を正しく選ぶ

この章で学ぶこと：整数と浮動小数点、文字列、変換時の失敗を整理する。

Nat は非負の自然数、U32 は固定幅の非負整数、F32 は近似的な浮動小数点数です。サイズの上限、演算結果、オーバーフロー、丸めが同じではありません。型が違う数をそのまま加算できるとは限らず、U32.to_nat など明示的な変換を使います。

String には split、join、trim、starts_with、to_upper などがあり、Char には文字に関する操作があります。単純な文字列の切り分けと、符号化や文字境界の意味を伴う操作は区別します。文字列を整数として読む U32.read は失敗し得るため、返り値の Maybe を確認します。

2.1 文字列の前処理と安全な変換

ユーザーが入力した文字列には、前後に空白が付いていることがあります。最初に String.trim で前後の空白を取り除き、次に U32.read で非負整数に変換する例を動かしましょう。

サンプル：空白を除去して文字列を数値に変換する

```
import Base

# String.read-like conversion can fail. Whitespace is removed first.
def parsed_number(value: Maybe<&2, U32>) -> U32:
  match value:
    case Some{n}:
      n
    case None{}:
      0

def main() -> U32:
  cleaned = String.trim(" 42 ")
  parsed_number(U32.read(cleaned))
```

U32.read は Maybe 型を返します。成功は Some、失敗は None です。このサンプルでは挙動を確認しやすいよう、失敗時に 0 を返しますが、**実際のアプリでは「失敗した 0」と「入力された 0」を混同しない設計**にしましょう。変換結果を Maybe のまま呼び出し元へ渡す方が明確です。

String.split は区切り文字を Char 一文字で受け取り、文字列のリストを返します。String.join はその逆方向の処理です。split と join の名前だけを覚えるのではなく、**区切り文字の型と、空入力のときの結果**を API と実行例で確認してください。保存ファイルの 1 行を String.split で分解する場合も、項目数と数値変換の失敗を調べる必要があります。

Cmp は大小の比較結果を表し、Bool は真偽を表します。「大小の三種類」と「条件式の二種

類」を使い分けると、ソートや検索の API を読みやすくなります。

演習　文字列`"42"`と`"x"`を数値として読む例を作り、成功・失敗の結果の型と中身を説明してみましょう。

Chapter 3

List と Array の詳しい使い方

この章で学ぶこと：構造の違いと、関数の選択が所有権に与える影響を理解する。

`List` は `Nil` と `Con` からなる再帰的な鎖です。`map`、`filter`、`foldl`、`foldr`、`sort` などが用意されています。変換したリストを作るとき、入力要素の型が `Type` か `Data` かによって、要素を何度も使えるかが変わります。

サンプル：リストを作り要素数を求める

```
import Base

# A linked list, and its length computed by the Base library.
def main() -> Nat:
  xs = List.replicate(U32, 3n, 7)
  List.length(&2, U32, xs)
```

このプログラムは `List.replicate` で値を 3 個並べたリストを作り、その長さを返します。`List.length` の引数には、数量と要素型が必要です。先頭への追加には `<>` という記法も使えます。

3.1 List.filter と foldl を組み合わせる

`List.filter` は条件を満たす要素だけを残します。`List.foldl` は先頭から値を集計し、最後の一つの値にまとめます。「抽出」と「集計」は別の操作です。

サンプル：小さい整数だけを抽出して合計する

```
import Base

# List.range returns List<&2, Nat>. Filter keeps that quantity.
def small(n: Nat) -> Bool:
  Nat.is_lt(n, 3n)

def sum(acc: Nat, n: Nat) -> Nat:
  Nat.add(acc, n)

def main() -> Nat:
  xs = List.range(4n)
  ys = List.filter(~Nat, ~small, xs)
  List.foldl(~&2, ~Nat, ~Nat, ~sum, ys, 0n)
```

`List.range(4n)` は 0 から 3 までのリストを作り、`List.filter` で 3 未満の値を残します。その合計は `3n` です。`foldl` は「現在までの合計」と「次の要素」を受け取る関数を引数にします。

List.filter は List<&2, A> を扱う一方、List.map は List<A> を受け取るなど、同じ List に見えても数量指定が異なる API があります。呼び出せないときは、引数の個数だけでなく**型の数量指定**まで比較しましょう。型の制約に合わないリストを強引に再解釈するのではなく、選ぶ関数やデータの構造を見直します。

Array はインデックスを使う構造です。get と set のほか、swap、clone、fork、join、to_list、map などがあります。「読むだけ」のつもりでも所有権付き配列を消費するので、返された配列を引き継ぎます。複製やフォークは処理・メモリのコストを伴います。

3.2 Array.new の引数は長さではない

Array.new の第二引数は「作る要素数」そのものではなく、二分木状の配列の深さです。深さを d とすると要素数は 2^d です。たとえば Array.new(U32, 3n, 0) は 8 要素を作ります。

サンプル：8 要素の配列の 2 番目と 7 番目を書き換える

```
import Base

# Array.new(d) produces an array of 2^d elements.
def read_and_sum(r: Array<U32> & U32) -> U32:
  (a, x) = r
  x

def main() -> U32:
  a = Array.new(U32, 3n, 0)
  b = Array.set(U32, a, 2, 17)
  c = Array.set(U32, b, 7, 25)
  x = read_and_sum(Array.get(U32, c, 2))
  x
```

Array.set は更新した配列を返します。以前の配列をそのまま後続で使うのではなく、返された配列を引き継ぎます。Array.get は配列と読み出した値の組を返します。例では配列を再使用しないため捨てていますが、次の読み出しや更新を行う場合は受け取ってください。インデックスが範囲外の場合の扱いも、公開 API の実装を確認しましょう。

演習　同じ数値列を List と Array の両方で表し、要素の変更、走査、複製が必要な場合の選び方を比較してみましょう。

Chapter 4

Map と Set を使う

この章で学ぶこと：辞書と集合の操作、キーの制約、更新後の所有権を理解する。

`Map` は**文字列をキーとする辞書**です。`new`、`set`、`get`、`has`、`del`、`keys`、`size` などがあり、値の型と数量を指定します。位置を表すキーを`"2,3"` のように構成すれば、盤面の座標を覚えることもできます。

サンプル：Map に記録を保存して読み取る

```
import Base

# Map.get returns a rebuilt map and its value.
def take_value(r: Map<&2, U32> & U32) -> U32:
  (again, value) = r
  value

def main() -> U32:
  m0 = Map.new(&2, U32)
  m1 = Map.set(&2, U32, m0, "wins", 7)
  take_value(Map.get(U32, 0, m1, "wins"))
```

`Map.get` には存在しないキーに対して使う既定値が渡されます。欠落と既定値を区別したい場合は `has` や別の API を組み合わせます。

`Set` は重複を持たない文字列キーの集合です。`add`、`has`、`del`、`size`、`to_list` などを使います。`Set.has` の結果に集合が含まれるのは、アフィン性に従って同じ集合を勝手に二度使わないためです。

サンプル：集合への追加と所属検査

```
import Base

# Set contains String keys; membership returns the set as well.
def membership(r: Set() & Bool) -> U32:
  (again, exists) = r
  match exists:
    case True{}:
      1
    case False{}:
      0

def main() -> U32:
  s0 = Set.new()
  s1 = Set.add(s0, "2,3")
  membership(Set.has(s1, "2,3"))
```

サンプル：同じ座標の旗を二度切り替える

```
import Base

# Keyed squares: Set operations are affine, so the new set is passed on.
def from_membership(r: Set() & Bool, key: String) -> Set():
  (s, present) = r
  match present:
    case True{}:
      Set.del(s, key)
    case False{}:
      Set.add(s, key)

def toggle_mark(s: Set(), +key: String) -> Set():
  from_membership(Set.has(s, key), key)

def main() -> Nat:
  s0 = Set.new()
  s1 = toggle_mark(s0, "2,3")
  s2 = toggle_mark(s1, "2,3")
  Set.size(s2)
```

後半の例は、旗を置く場所の集合を表します。旗操作は `Set.has` で確認し、存在すれば削除、なければ追加します。マインスイーパー本体の盤面管理に使える小さな部品です。

4.1 Map.keys と Set の削除

辞書や集合は値を追加するだけではありません。すでに存在するキーを数え、列挙し、削除する操作も頻繁に使用します。

サンプル：重複追加・削除・キーの列挙

```
import Base

# Set.add discards duplicates; Map.keys returns a list of String keys.
def main() -> Nat:
  s0 = Set.new()
  s1 = Set.add(s0, "0,0")
  s2 = Set.add(s1, "0,0")
  s3 = Set.add(s2, "1,1")
  s4 = Set.del(s3, "0,0")
  m0 = Map.new(&2, U32)
  m1 = Map.set(&2, U32, m0, "wins", 3)
  m2 = Map.set(&2, U32, m1, "games", 7)
  keys = Map.keys(&2, U32, m2)
  a = Set.size(s4)
  b = List.length(&2, String, keys)
  Nat.add(a, b)
```

ここでは `Set` に同じキーを二度追加しても件数は増えないことを確認します。その後、追加済みのキーを削除します。一方、`Map.keys` は辞書の文字列キーのリストを返すので、`List.length` で数えられます。

`Map.keys` で得られたリストの順番に、登録順の保証を勝手に想定してはいけません。表示順が必要なら、明示的にソートします。`Map.get` は既定値を返せますが、値に 0 が格納されている場合は「存在しない」と区別できません。`Map.has` を組み合わせるか、値の型に `Maybe` 等を使う設計を検討しましょう。

演習　存在しないキーを `Map.get` したときの結果、同じ座標を `Set.add` したときのサイズを

確かめましょう。

Chapter 5

失敗、選択肢、依存する値

この章で学ぶこと：Maybe、Result、Either、Pair、Sigma などの役割を分ける。

`Maybe<T>` は値がない可能性を表します。`Result<E,T>` は失敗理由を含めて返す際に使います。`Either` は二つの異なる型のいずれかを表し、`Pair` は二つの値を一緒に返します。`Sigma` は二番目の値の型が一番目の値に依存する組を表すときに登場します。

単に `0` を失敗扱いする設計だと、0 が正常な結果である場合に曖昧になります。見つからない座標を表すなら `Maybe`、保存ファイルの読み込みに失敗するならエラー理由を持つ `Result` が適切です。

`Kind` と `Quant` は、型の値を複製できるか、引数を消去できるかなどを制約します。配列・ファイルハンドル・関数・値の複製を同じように考えないことが重要です。次の Part ではファイルハンドルを例に確認します。

演習　「座標が存在しない」「ファイルを開けない」「2 つの集計値を返す」に、それぞれどの結果型がふさわしいか説明してみましょう。

Part II

入出力と並行処理

Chapter 6

IO を読む

この章で学ぶこと：純粋な計算と外部への作用を分け、失敗を安全に伝える。

`IO<A>` は、外部とのやり取りを行って `A` 型の結果を得る計算を表します。`IO.print` は画面出力、`IO.args` は起動引数、`IO.get_env` は環境変数を扱います。`do IO<...>` では処理の順序を明示できます。

`IO.try` は `Result` として返された成功値を取り出し、失敗を IO の失敗へ伝えます。`IO.die` は失敗終了を表します。どこで失敗し得るのかを把握し、入力の不正と環境の失敗を区別するのが目標です。

マインスイーパーでは「地雷をクリックした」は正常なゲーム上の結果ですが、「保存先へ書けない」は IO 上の失敗です。両者を同じ種類の例外として扱わない設計が読みやすくなります。

演習　ファイルがないことと、ゲームに負けたことを別の型で表す理由を説明してみましょう。

Chapter 7

File とリソースの受け渡し

この章で学ぶこと：ファイルの開閉、文字・バイト列、失敗後の扱いを学ぶ。

`File.open(path,mode)` は `IO(Result<...,File>)` を返します。開けないときはハンドルがありません。`File.read(file,max)` は、**継続して使う File と読み込み結果の組**を返します。書き込み操作も同様に結果を確認します。最後に `File.close` を呼びます。

サンプル：ファイルへ文字列を書き込む例

```
import Base

# File operations return the handle so it can be used or closed exactly once.
# Run in a writable directory. This writes "sample_board.txt".

def close_after_write(r: File & Result<&1, &1, U32 & String, Unit>) -> IO(Unit):
  (file, status) = r
  match status:
    case Done{value}:
      File.close(file)
    case Fail{error}:
      do IO<Unit>:
        Unit <- File.close(file)
        IO.die(Unit, 1, "Could not save sample_board.txt")

def main() -> IO(Unit):
  do IO<Unit>:
    file: File <- IO.try(File, File.open("sample_board.txt", "w"))
    written: File & Result<&1, &1, U32 & String, Unit> <- File.write(file,
     "board-size=4\nmines=1,1\n")
    close_after_write(written)
```

この例は作業ディレクトリに `sample_board.txt` を作ります。ファイルを開けないときには `IO.try` が処理を失敗として終了します。`File.write` は元のファイルハンドルを消費し、書き込み結果と**継続して利用できるハンドル**を返します。`close_after_write` は `Done`・`Fail` を分け、どちらもハンドルを閉じます。

ファイルへのアクセスは、成功した書き込みだけを想定してはいけません。開くときの権限不足、保存先の不足、書き込みの失敗などを検査します。保存後にファイルを読み戻すと、期待した内容が書き込まれたか確かめられます。

7.1 File.read による読み込みとラウンドトリップ

保存の次は読み込みです。`File.open(path, "r")` で読み取り用に開き、`File.read(file, 4096)` で最大 4096 バイトを受け取ります。読み込みの結果は `Result` に包まれており、成功時の `String` とファイルハンドルを取り出してから、必ず `File.close` します。

サンプル：保存した文字列を読み取る例

```
import Base

# Reads the file written by 12_file_write.bend.
# Every branch closes the affine File handle.

def close_after_read(r: File & Result<&1, &1, U32 & String, String>) -> IO(String):
  (file, status) = r
  match status:
    case Done{contents}:
      do IO<String>:
        Unit <- File.close(file)
        return contents
    case Fail{error}:
      do IO<String>:
        Unit <- File.close(file)
        IO.die(String, 1, "Could not read sample_board.txt")

def main() -> IO(Unit):
  do IO<Unit>:
    file: File <- IO.try(File, File.open("sample_board.txt", "r"))
    result: File & Result<&1, &1, U32 & String, String> <- File.read(file, 4096)
    contents: String <- close_after_read(result)
    IO.print(contents)
```

この例は `12_file_write.bend` の実行後に動かします。ファイルが存在しなければ読み込みは失敗として終了します。保存した内容を読み戻して元の文字列と比較すると、保存・復元を一組としてテストできます。これを**ラウンドトリップテスト**と呼びます。

バイト列を直接扱う場合には、次の別の操作を使います。

- `File.read_bytes`：バイト列を読み込みます。
- `File.write_bytes`：バイト列を書き込みます。

`File.read` が一度の呼び出しで必ずファイル全体を返すとは限りません。大きいファイルでは残りを繰り返し読む設計が必要です。文字列とバイト列、ファイルの終端、指定した読み込み最大長の関係も API の型から確認しましょう。

保存形式には、まず単純な行単位のテキストを使うとよいでしょう。地雷位置、開いたマス、旗位置、盤面サイズ、版番号を別々に記録し、読み込み時に入力の妥当性をチェックします。任意の外部ファイルを信頼してはいけません。

演習　「存在しないファイル」「空ファイル」「項目が欠けたファイル」の三種類を用意し、それぞれどう扱うべきか設計してみましょう。

Chapter 8

引数、時刻、乱数、プロセス

この章で学ぶこと：外部情報を受け取る方法と、再現できるテストの作り方を学ぶ。

`IO.args` は起動時の引数をリストとして受け取ります。設定ファイルの位置や盤面のサイズを指定する例で使えます。`IO.get_env` は環境変数を読みます。

サンプル：引数の個数を表示する

```
import Base

# IO.args returns a list including the executable or source path.
def main() -> IO(Unit):
  do IO<Unit>:
    xs: List<String> <- IO.args()
    IO.print(Nat.show(List.length(&1, String, xs)))
```

8.1 環境変数は常に存在するとは限らない

環境変数は OS や起動方法によって変化します。`IO.get_env("PATH")` はパス検索の設定を読む例ですが、必ず値があると仮定してはいけません。

サンプル：環境変数を読み取り長さを表示する

```
import Base

# The environment may not contain the requested name.
def env_length(value: Result<&1, &1, U32 & String, String>) -> Nat:
  match value:
    case Done{s}:
      String.length(s)
    case Fail{problem}:
      0n

def main() -> IO(Unit):
  do IO<Unit>:
    result: Result<&1, &1, U32 & String, String> <- IO.get_env("PATH")
    IO.print(Nat.show(env_length(result)))
```

`IO.get_env` の結果は `Result` です。`Done` の場合は文字列を受け取り、`Fail` の場合は例のように既定動作へ分岐できます。エラー内容が必要なら、`Fail` の値を捨てずに利用します。環境変数の中身をそのままログに出すと、パスワードなどの機密情報を漏らす可能性があります。この例では値そのものではなく長さだけを出力します。

`IO.now` は現在時刻、`IO.sleep` は時間待ち、`IO.random_u32` は乱数を扱います。乱数を使ったテストは、実際の乱数に依存させず、固定した地雷配置を入力にする方法と分けて実施します。

8.2 乱数の取得と失敗の伝播

`IO.random_u32()` の型は `IO(Result<..., U32>)` であり、単に `U32` が返るわけではありません。外部機能が失敗する可能性を明示するためです。`IO.try` を使うと成功した `U32` を取り出せますが、失敗したときは後続の `IO.print` へは進みません。

サンプル：乱数を取得して表示する

```
import Base

# The random-number effect may fail, hence the Result inside IO.
def main() -> IO(Unit):
  do IO<Unit>:
    random_value: U32 <- IO.try(U32, IO.random_u32())
    IO.print(U32.show(random_value))
```

この例は毎回値が変化するため、数値そのものを固定して比較するテストには不向きです。ソルバーの正しさは固定した盤面でテストし、ランダム盤面の生成はその後に確認しましょう。

8.3 外部プログラムを起動する

`Process.run` は外部プログラムを起動します。引数のリスト、標準入力へ渡す文字列、受け取り可能な出力の最大量、タイムアウトを指定します。

サンプル：外部プログラムを起動して終了コードを確認する

```
import Base

# Launch a harmless OS program and examine its exit code.
def exit_status(value: U32 & (String & String)) -> U32:
  (code, outputs) = value
  code

def main() -> IO(Unit):
  do IO<Unit>:
    result: U32 & (String & String) <- IO.try(U32 & (String & String),
    Process.run("/usr/bin/true", Nil{}, "", 1024, 3000))
    IO.print(U32.show(exit_status(result)))
```

この例は Unix 系環境の `/usr/bin/true` を実行します。このコマンドは何も表示せずに成功の終了コード 0 を返します。Windows などでは同じパスは使えないため、実行環境に応じてプログラムを選んでください。

`Process.run` は `IO(Result<...>)` 型の結果を返します。プロセスの**起動そのものが失敗した場合**と、**起動したプログラムが 0 以外の終了コードで終わった場合**は違います。前者は `Fail` で、後者は成功結果内の終了コードで区別します。返り値には標準出力と標準エラー出力も含まれます。ここでは終了コードだけを取り出しますが、利用者に必要な情報を捨ててはいけません。

演習　引数が不足するときは既定の盤面を使うか、理由を表示して終了するか、用途に応じて選んでみましょう。

Chapter 9

IO.fork、Chan とタイムアウト

この章で学ぶこと：独立した仕事を非同期に実行し、結果の受信と終了処理を理解する。

CPU の二つの関数呼び出しを並列に実行する記法と、IO 処理を並行させる操作は別です。`IO.fork` は IO を開始して結果を受け取るチャネルを返し、`IO.join` が結果を待ちます。`Chan.new`、`Chan.send`、`Chan.recv` はメッセージの受け渡しに使います。

`Chan.recv` は閉じられたチャネルから値が得られない場合、`None` を返すことがあります。`IO.within` など時間制限を扱う API は、時間内に完了しなかった場合の結果を確認して使います。待ち手だけを途中で止めたとき、仕事がどうなるかも仕様と実装で確認する必要があります。

9.1 独立した IO を開始して結果を待つ

`IO.fork` は IO 計算を始め、その結果を受け取るための `Chan` を返します。`IO.join` はその結果を待ち、チャネルを閉じます。最も小さな例では、別の IO 計算が返す値を受け取るだけでも利用できます。

サンプル：IO.fork で開始し IO.join で結果を受け取る

```
import Base

# fork returns a channel for the result; join waits and closes it.
def main() -> IO(Unit):
  do IO<Unit>:
    chan: Chan(U32) <- IO.fork(U32, IO.pure(U32, 7))
    value: U32 <- IO.join(U32, chan)
    IO.print(U32.show(value))
```

この例の結果は 7 です。これだけでは、CPU が別スレッドで実際に動いたことや処理が高速化したことを意味しません。`IO.pure` は、外部作用を起こさずに値を IO 計算の結果とする関数です。

`Chan.send` の結果は、送信できた場合の `Done` か、チャネルが閉じた場合の `Fail` です。`Chan.recv` は値が届いた場合の `Some` か、閉じた後で受信できない場合の `None` を返します。`IO.join` が閉じられたチャネルから値を受け取れない場合は失敗終了します。適切に結果を待ち、チャネルを閉じる責任を曖昧にしないようにしましょう。

`IO.within` は、指定時間内に結果が得られたかを `Maybe` として返します。注意すべき点として、タイムアウトしたからといって**遅れた仕事が自動的に取り消されるわけではありません**。処理が止まらない可能性がある用途では、個別の終了・中断方法も検討します。

この章では操作の型と意味を独立例で学び、マインスイーパーでは自動保存やタイマーの用

途と結び付けます。最初から複雑なワーカープールを作る必要はありません。

演習　送る側が終了したとき、受信する側はいつ処理を終えるべきか、短い手順を書いてみましょう。

Chapter 10

TCP、UDP と外部環境

この章で学ぶこと：接続の確立、失敗、リソース開放を理解する。

`TCP.listen`、`TCP.accept`、`TCP.connect`、`TCP.send`、`TCP.recv` は接続型の通信です。`UDP.bind`、`UDP.send_to`、`UDP.recv_from` はデータグラムをやり取りします。通信には相手が応答しない場合や、受信データが途中で切れる場合があります。

どちらも `Result` を調べ、`Socket` や `Listener` の所有権を返り値で受け渡します。ローカルホストの最小例から始め、外部ネットワークへの接続は別途許可を得た環境で行います。`Process.run` や音声・ファイル IO と同様、ホスト OS で使用できる機能に影響されます。

通信はマインスイーパーへ必須ではありません。本書では「何ができ、どの型を返し、どこで失敗するか」を独立した学習事項とします。

演習　TCP と UDP で「送信成功」の意味が異なるのはなぜか、通信が届かない場合を考えて説明してみましょう。

Part III

Image・App・GUI

Chapter 11

Image と四分木

この章で学ぶこと：Pix と Qua による画像表現を理解する。

Bend 2 の `Image` は再帰的な画像型です。`Pix` は一色の正方形、`Qua` は左上・右上・左下・右下に画像を配置して大きな正方形を作ります。小さな画素を一個ずつ描く方法だけでなく、同じ色の領域をまとめて表現できます。

```
Pix{0x002244}
Qua{Pix{0}, Pix{0xffffff}, Pix{0xffffff}, Pix{0}}
```

上記は画像値の構成例です。RGB 値の解釈や、表示するウィンドウのサイズと四分木の深さの関係は、実行環境で確かめます。マインスイーパーでは、閉じたマス、開いたマス、旗、地雷を別々の色・形で描画します。

演習　同じ色が広く続く背景を `Pix` 一個で表す利点を説明してみましょう。

Chapter 12

App とイベントの処理

この章で学ぶこと：view と tick を分離し、状態を画面へ描く。

App は `view` と `tick` の組です。`view` は現在の状態を受け取り、状態と画像の組を返します。`tick` は `List<Event>` を受け取り、次の状態を `Maybe` で返します。`None` は終了を意味します。`Event` にはマウス・キー・スクロール・閉じる操作などがあります。

サンプル：イベントで色を切り替える最小 GUI

```
import Base

# Adapted from the official GUIDE. Requires a graphical desktop to run.
def tick(events: List<Event>, color: U32) -> IO(Maybe<U32>):
  match events:
    case Nil{}:
      IO.pure(Maybe<U32>, Some{color})
    case Con{Close{}, rest}:
      IO.pure(Maybe<U32>, None{})
    case Con{e, rest}:
      tick(rest, (color + 1 : U32))

def main() -> IO(Unit):
  App.run(~U32, ~App{+s => (s, Pix{s}), tick}, "GUI example", 256, 256, 0)
```

この例は公式ガイドの構成に沿ったものです。画面を開ける環境で動かしてください。型検査の成功だけでは、ウィンドウが表示できることまでは保証されません。

GUI アプリで重要なのは、入力イベントを受けたときに**純粋な状態更新関数を先に呼び、その結果を描画する**ことです。そうすればイベントそのものを再現しなくても、更新の正しさをテストできます。

演習　`Close` が来たらアプリを終了する処理と、それ以外の入力を受けたときの処理を区別してみましょう。

Chapter 13

マインスイーパーの状態

この章で学ぶこと：地雷配置、表示状態、操作を独立させる。

マインスイーパーのマスには「まだ開いていない」「旗が付いている」「開いていて周囲の地雷数が分かる」といった状態があります。内部の地雷の有無と、プレイヤーに見える状態は分ける必要があります。

サンプル：セル状態と旗の切替

```
import Base

# The local rules are pure. A later chapter attaches them to a full board.
type Square is Data:
  Hidden{}
  Flag{}
  Revealed{nearby: U32}

def flip_flag(s: Square) -> Square:
  match s:
    case Hidden{}:
      Flag{}
    case Flag{}:
      Hidden{}
    case Revealed{count}:
      Revealed{count}

def is_flag(s: Square) -> U32:
  match s:
    case Flag{}:
      1
    case Hidden{}:
      0
    case Revealed{count}:
      0

def main() -> U32:
  is_flag(flip_flag(Hidden{}))
```

このプログラムでは、旗を付ける・外すという操作の入口を実装しました。本格的な盤面では、盤面の大きさ、地雷集合、公開済み集合、旗集合、ゲーム終了状態が必要です。

13.1 Set を利用した 8 近傍の数え上げ

次に、マインスイーパーの数字を決める部分を作ります。幅と高さを 4 とした小さな盤面で、地雷を Set 内の文字列"x,y" で管理します。あるマスの周囲 8 か所を調べ、地雷がある場所を 1、ない場所を 0 として足します。

サンプル：四隅の境界を考慮した周囲 8 マスの地雷数

```
import Base

# A 4x4 Minesweeper board; mine positions are String keys "x,y".
# Out-of-bounds neighbor coordinates must NOT wrap around the board.

def member_bit(r: Set() & Bool) -> U32:
  (s, found) = r
  match found:
    case True{}:
      1
    case False{}:
      0

def mine_at(mines: Set(), x: U32, y: U32) -> U32:
  member_bit(Set.has(mines, U32.show(x) ++ "," ++ U32.show(y)))

def guarded(+mines: Set(), x: U32, y: U32, inside_x: Bool, inside_y: Bool) -> U32:
  match inside_x inside_y:
    case True{} True{}:
      mine_at(mines, x, y)
    case _ _:
      0

def adjacent(+mines: Set(), +x: U32, +y: U32) -> U32:
  +left = (x - 1 : U32)
  +right = (x + 1 : U32)
  +up = (y - 1 : U32)
  +down = (y + 1 : U32)
  a = guarded(mines, left, up, U32.is_lt(left, 4), U32.is_lt(up, 4))
  b = guarded(mines, x, up, U32.is_lt(x, 4), U32.is_lt(up, 4))
  c = guarded(mines, right, up, U32.is_lt(right, 4), U32.is_lt(up, 4))
  d = guarded(mines, left, y, U32.is_lt(left, 4), U32.is_lt(y, 4))
  e = guarded(mines, right, y, U32.is_lt(right, 4), U32.is_lt(y, 4))
  f = guarded(mines, left, down, U32.is_lt(left, 4), U32.is_lt(down, 4))
  g = guarded(mines, x, down, U32.is_lt(x, 4), U32.is_lt(down, 4))
  h = guarded(mines, right, down, U32.is_lt(right, 4), U32.is_lt(down, 4))
  (a + b + c + d + e + f + g + h : U32)

def main() -> U32:
  +mines = Set.add(Set.new(), "1,1")
  a = adjacent(mines, 0, 0)
  b = adjacent(mines, 2, 2)
  (a + b : U32)
```

Set.has は集合と Bool の組を返すので、member_bit で真偽を 0 か 1 に変えます。複数回の検索をするため、Set() が Data であることを利用し、+mines で明示的に再利用します。

重要なのは**端の外側を反対側へつながないこと**です。前巻のライフゲームはトーラス境界でしたが、ここでは盤面の外を地雷なしとして数えます。座標の範囲を判定する guarded はこのためにあります。例では地雷を (1,1) に一つ置き、(0,0) と (2,2) の数字が両方 1 になることを確かめます。main の返り値はそれらの合計 2 です。

演習 地雷を (0,0) に動かし、(0,1)、(1,1)、(3,3) の数字を予想して、実行結果と比較してみま

しょう。

最初のクリックで地雷を踏まない規則、0 のマスの周囲を自動で開く規則、勝敗条件は、ゲームの状態遷移として定義します。`law` で検査する性質と、GUI で操作して確かめる性質を区別します。

演習 公開済みのマスに旗を立ててよいかどうか、先にゲームの仕様として決めてから関数を設計してみましょう。

Chapter 14

表示と入力をテストする

この章で学ぶこと：GUI がなくても検証できる部分を特定する。

描画結果を生成する関数は、同じ盤面から同じ `Image` を構成すべきです。マウスの座標から盤面のインデックスを求める関数も純粋にできます。入力範囲外、ウィンドウ端、境界上などをテストします。

一方、マウス装置が実際に動くか、ウィンドウを作成できるか、描画した色が正しく表示されるかは環境に依存します。純粋関数の動作は自動テストで確かめられますが、ウィンドウの操作は実際の画面上でも確認します。`Window` の直接操作や `Audio` の使用も、基本の `App` を理解した後の独立演習にします。

演習　マスの幅を 32 画素、盤面を 8 列としたとき、マウス座標とマス座標の変換式を求めてみましょう。

Part IV

仕様駆動開発と形式的証明

Chapter 15

law を先に書く

この章で学ぶこと：満たしたい性質を文章と式で定義し、実装と証明を区別する。

例えば旗の切替は「閉じたマスは旗付きになる」「旗付きは閉じたマスに戻る」「開いたマスはそのまま」という仕様です。これに加え「二回切り替えると元に戻る」という法則を作れます。

サンプル：旗操作に関する法則と証明

```
import Base

type Cover is Data:
  Hidden{}
  Marked{}
  Open{}

def toggle(c: Cover) -> Cover:
  match c:
    case Hidden{}:
      Marked{}
    case Marked{}:
      Hidden{}
    case Open{}:
      Open{}

law toggle_twice:
  for c: Cover
  {toggle(toggle(c)) == c : Cover}

def toggle_twice(c):
  match c:
    case Hidden{}:
      {==}
    case Marked{}:
      {==}
    case Open{}:
      {==}

law opened_stays_open:
  {toggle(Open{}) == Open{} : Cover}

def opened_stays_open():
  {==}
```

`law` は普遍的に成り立つと主張する命題、同名の `def` はその証明です。`for` は対象となる値の範囲を指定します。`match` を使う場合、取り得るコンストラクタごとに証明します。

重要：「二回で戻る」だけだと、何もしない関数も満たしてしまいます。仕様は弱すぎてもいけません。期待する振る舞いを漏れなく命題として表す必要があります。

演習 旗を付けた後の状態が `Marked` になる性質を別の `law` で定義してみましょう。

Chapter 16

等式、帰納法、補題と書き換え

この章で学ぶこと：法則を小さく分割し、複数の命題を組み合わせる。

`Equal.sym` は等式の向きを入れ替え、`Equal.trans` は等式をつなぎます。帰納的なデータに対しては、空のケースと再帰的なケースを分けて証明します。再帰の停止性が必要なのは、停止しない「証明」を受け入れると矛盾する命題まで証明できてしまうためです。

`Sigma`・`Exists` などは、値の存在とその性質を一緒に表すのに使えます。`@unsafe` や外部実装を含む関数は、純粋な範囲と同じ保証を持つわけではありません。型が通った、サンプルが動いた、定理が証明できた、という三つの主張は常に分けてください。

```
bend intermediate/src/04_flag_laws.bend --verdict
```

上の命令で証明検査の結果を確認できます。`?TODO` などの穴を残したものを、証明完了として採用してはいけません。

演習　二つの等式を順番につなぐ場合、何を中間式として選ぶか考えてみましょう。

Chapter 17

盤面の不変条件

この章で学ぶこと：局所的な操作の法則から盤面全体の仕様へ進む。

盤面全体の仕様を考えるとき、地雷位置の集合は操作によって変化しない、公開済み集合は通常の公開操作では減らない、表示する数字は周囲 8 マスの地雷数に等しい、などを区別します。

$$\mathrm{mines}(\mathrm{reveal}(b,p)) = \mathrm{mines}(b), \qquad \mathrm{open}(b) \subseteq \mathrm{open}(\mathrm{reveal}(b,p)).$$

これらは**目指す仕様**であり、この初稿で盤面全体を証明済みという意味ではありません。実装後、座標変換、隣接位置の列挙、公開操作、勝敗判定のそれぞれの補題を作る必要があります。

仕様を `LAWS.bend`、実装を別ファイル、証明を `PROOF.bend` に分けると、どこが要求でどこが検証かを整理できます。最初は同じファイルで小さな法則を試し、慣れてから分離します。

演習　一手目が安全であることを命題として書くとき、どんな入力と結果が必要か説明してみましょう。

Chapter 18

証明の範囲と信頼境界

この章で学ぶこと：保証できた性質と保証できなかった性質を正確に報告する。

地雷数の局所規則を証明しても、GUI が正しいマスを選ぶ保証や、保存データが必ず復元できる保証にはなりません。ファイル・ネットワーク・乱数・GPU・FFI など外部環境を含む部分は、純粋計算の形式的証明と別の検証が必要です。

依存型は入力に関する性質を型で表現する助けになりますが、「型注釈があるから必ず正しい」わけではありません。`law` で何を主張し、証明がどの仮定を使ったのかを確認します。断定できる範囲だけを読者へ伝えることが重要です。

演習　「盤面の地雷数が一定」と「ファイルへの保存が必ず成功する」が異なる種類の性質である理由を説明してみましょう。

Part V

並列実行とその他の応用

Chapter 19

CPU、GPU、Array.atomic

この章で学ぶこと：計算並列と入出力の並行を区別し、データの共有条件を理解する。

Bend の並列呼び出しでは、二つの独立した処理を同時に始める記法を使います。負荷が片方に偏ると待ち時間が増えます。

サンプル：均等な二分木での並列呼び出し

```
import Base

# Independent branches form a balanced fork/join computation.
def count_leaves(+level: Nat) -> U32:
  match level:
    case 0n:
      1
    case 1n+k:
      x y = count_leaves(k) count_leaves(k)
      (x + y : U32)

def main() -> U32:
  count_leaves(8n)
```

一方、関数呼び出しに! を付ける方法は GPU 側の実行指定です。環境によっては CPU で実行されます。必ず高速化するという意味ではありません。

`Array.atomic.add`、`Array.atomic.cas` などは共有されるデータの原子的更新に関わる API です。戻り値、対応する要素型、対象実行環境を公式 `Base` で確認してから使います。高速化の必要がないコードに無理に原子的操作を追加しません。

演習　独立した複数の盤面を並列に解析する場合、どの入力を分離する必要があるか考えてみましょう。

Chapter 20

C・JavaScript と外部連携

この章で学ぶこと：生成するプログラムとホスト環境の境界を知る。

Bend 2 は C のネイティブ実行ターゲットと JavaScript の実行ターゲットを持ちます。GPU やスレッドの振る舞い、使用できる外部機能はターゲット間で同じとは限りません。外部の C・JavaScript 実装を読み込む仕組みを使えば、Bend だけでは用意されていない操作を追加できます。

ただし、外部関数が返す結果や副作用は Bend 内の純粋な証明だけで保証できません。`@unsafe` などの安全性に関する境界とともに理解します。モジュールやパッケージの整理、ライセンス、BendHub 等を使った配布は、まず公式の最新ガイドで対応状況を確かめて進めます。

`Audio.open`、`Audio.write`、`Audio.close` も独立例として API の型を確認し、実際の音声出力は環境対応を確かめた場合だけ成功と記録します。

演習　同じサンプルを C ターゲットと JavaScript ターゲットで動かす際、何を別々に検証する必要があるか列挙してみましょう。

Part VI

マインスイーパー統合と発展

Chapter 21

一人で遊べるアプリにする

この章で学ぶこと：仕様、データ、画面、保存、テストを順番に統合する。

完成する通常版は、地雷配置、左クリックでの公開、右クリックでの旗の切替、0 マスの連鎖公開、勝敗判定、経過時間、再スタートを備えます。一手目が安全である条件も明示します。

1. 地雷配置と公開情報を別の型で保持する。
2. 公開、旗、勝敗判定を純粋関数で実装し、局所法則を書く。
3. `Image` へ盤面を描画し、`Event` から操作へ変換する。
4. 設定・戦績を `File` へ保存し、復元時は妥当性を検査する。
5. CLI の独立した期待値テストで規則を検証し、実機 GUI テストを追加する。

実際に統合する際は、一つずつ機能を追加し、テストで正しさを確認してから次の機能へ進めましょう。

演習　最初のクリック前の盤面と、地雷を配置した後の盤面を別の型として扱う利点を考えてみましょう。

Chapter 22

発展：運ゲー無しマインスイーパー

この章で学ぶこと：「公開情報から確実に分かること」を形式化し、ソルバーによる盤面検証と生成を理解する。

22.1　なぜ通常の盤面では推測が必要になるのか

通常のマインスイーパーでは、開いたマスに書かれた数字が同じでも、隠れている地雷の置き方が複数あり得ます。例えば、数字の 1 の周囲に未確定のマスが二つ残り、そのどちらにも地雷が置ける場合、片方を適当に開くことは推論ではありません。

運ゲー無し版では、**最初に開くマスが決まったあと、確実に安全と判定できる手だけを続けて勝利できる盤面**を採用します。Simon Tatham の『Mines』は、この考え方を使った公開実装です。実際にソルバーを用意し、候補の地雷配置を解かせて検証する方法が知られています。

22.2　最小の推論規則を Bend 2 で書く

開かれたマスに表示された数字を c、周囲で地雷と**確定した**マス数を f、まだ確定していないマス数を u とします。

$$r = c - f$$

は、未確定マスに残っている地雷数です。$0 \le r \le u$ が満たされるとき、次の二つの規則が使えます。

- $r = 0$：周囲の未確定マスはすべて安全です。
- $r = u$：周囲の未確定マスはすべて地雷です。

$0 < r < u$ であれば、その数字だけでは決まりません。$r > u$ は矛盾です。プレイヤーが旗を置いたマスを無条件で「確定した地雷」にしてはいけません。誤った旗は誤推論を引き起こすためです。

サンプル：局所制約の判定と法則

```
import Base

# Deduce from ONE visible number:
# remaining = displayed clue - mines already established as certain
# unknown   = number of still-unclassified adjacent squares
#
# "Flagged by the player" is NOT automatically "certain mine".
```

```
type LocalRule is Data:
  AllSafe{}
  AllMines{}
  Unknown{}
  Inconsistent{}

# Definition order matters: each helper is available to those below it.
# Computed conditions are passed as function parameters to pattern matches.
def infer_equal(remaining: U32, unknown: U32, equal: Bool) -> LocalRule:
  match equal:
    case True{}:
      AllMines{}
    case False{}:
      Unknown{}

def infer_zero(+remaining: U32, +unknown: U32, empty: Bool) -> LocalRule:
  match empty:
    case True{}:
      AllSafe{}
    case False{}:
      infer_equal(remaining, unknown, U32.is_eq(remaining, unknown))

def infer_limit(+remaining: U32, +unknown: U32, over: Bool) -> LocalRule:
  match over:
    case True{}:
      Inconsistent{}
    case False{}:
      infer_zero(remaining, unknown, U32.is_eq(remaining, 0))

def infer(+remaining: U32, +unknown: U32) -> LocalRule:
  infer_limit(remaining, unknown, U32.is_lt(unknown, remaining))

# A small observable numeric output for automated testing.
def tag(r: LocalRule) -> U32:
  match r:
    case AllSafe{}:
      1
    case AllMines{}:
      2
    case Unknown{}:
      3
    case Inconsistent{}:
      4

law zero_remaining_is_safe_for_three:
  {infer(0, 3) == AllSafe{} : LocalRule}

def zero_remaining_is_safe_for_three():
  {==}

def main() -> U32:
  a = tag(infer(0, 3))
  b = tag(infer(3, 3))
  c = tag(infer(1, 3))
  d = tag(infer(4, 2))
  (a + b + c + d : U32)
```

`infer` は残り地雷数と未確定マス数から結果を表す ADT を返します。`AllSafe` や `AllMines` は推論の種類を表し、マスの座標自体を返すものではありません。`Unknown` の場合に勝手に安全マスを決めてはいけません。入力に矛盾があるときは `Inconsistent` で区別します。付属の `law` は、残り地雷数が 0 で未確定マスが 3 個ある場合の具体例です。任意のマス数や盤面全体

を証明したものではありません。

演習 未確定マスが 4 個、残り地雷数が 0、2、4、5 の場合に、それぞれ `infer` が何を返すべきか調べてみましょう。

22.3 複数の数字から推論する

一つの数字だけで決まらない場合でも、複数の数字が制約する未確定マスの集合を比較すると、情報が得られることがあります。集合 A に地雷が a 個、集合 B に地雷が b 個という制約があり、$A \subseteq B$ なら、

$$\#\big((B \setminus A) \cap M\big) = b - a$$

です。特に $b = a$ なら $B \setminus A$ のマスはすべて安全です。$b - a = |B \setminus A|$ ならすべて地雷です。この**部分集合の規則**は、`Set` の活用例でもあります。

さらに、残り地雷の総数を使った推論もあります。独立した場所に分かれる制約でも、総地雷数によって片方の候補が排除されることがあります。標準ライブラリの集合・リスト・辞書をどう使うか検討するのが中級編としての目的です。

22.4 公開情報と真の地雷配置を分離する

公開状態 V から矛盾なく置ける地雷配置の集合を $\mathcal{C}(V)$ とします。ここには**開いたマスとその数字、盤面サイズ、総地雷数**だけが反映されます。未公開の本当の地雷位置は渡しません。

$\mathcal{C}(V) \neq \varnothing$ であり、未公開マス p がすべての候補で地雷でないなら、

$$\forall M \in \mathcal{C}(V), \quad p \notin M,$$

p は確実に安全です。候補集合が空の場合は、入力が矛盾しているものとしてエラーにします。空集合に対する全称命題を利用して「安全」と判定するのは誤りです。

この区別をコードで守ることが重要です。

- **ソルバー**：公開状態 V だけを受け取り、確実な安全マスを返す。
- **ゲーム本体**：真の配置を保持し、実際に開いたマスの数字を計算する。
- **検証器**：ソルバーの提案した操作をゲーム本体で再生し、最後まで安全にクリアできるか検査する。

22.5 候補生成と検証を繰り返す

初手 p_0 を決め、そのマスを必ず安全にするよう候補の地雷配置を作ります。より大きく開ける開始局面にしたい場合は、初手とその周囲 8 マスを地雷禁止領域にできます。その後、ソルバーに**公開状態だけ**を渡して確実な操作を繰り返させます。

1. 初手が安全な地雷配置を生成します。
2. 初手を開き、数字が 0 なら周囲を連鎖的に開きます。
3. 公開数字と総地雷数から、確実な安全マスを探します。
4. 安全マスが見つかれば開き、全安全マスを開けるまで続けます。

5. 候補が残っているのに確実な手がなければ、その盤面は採用しません。

生成の試行回数には上限を設けます。上限に達した場合は**生成失敗を明示**し、推測が必要な盤面を無断で運ゲー無し版として公開してはいけません。あらかじめ検証した盤面を別途用意して代替する方法もあります。

22.6　小盤面用の参考実装と検証

この節の発展用参考実装はリポジトリの 中級編の参考実装にあります。学習用 ZIP には完成コードを入れず、読者が必要に応じて参照できます。

小盤面用の Python 検証器は、公開数字と総地雷数に矛盾しない候補配置を**全列挙**し、そのすべてで安全なマスだけを開きます。列挙数が上限を超えた場合は採用を保留します。これにより、限定した盤面サイズでは、局所規則で分からない確定手まで探せます。

ただし、候補の全列挙は盤面が大きくなると指数的に重くなります。これは中級編の学習・テスト用の独立した参考実装であり、実用サイズの Bend 版ソルバーや GUI アプリの完成を意味しません。Bend への移植を進める際は、まず局所推論と集合の差分から取り組みましょう。

22.7　「運ゲー無し」の保証範囲

解が一意であることと、途中の局面で確実な手を選べることは異なります。また、局所規則しか知らないソルバーが手詰まりになっても、もっと強い推論で解ける可能性があります。したがって、ソルバーが最後まで解けた盤面だけを採用する方法は確実ですが、採用されなかった盤面が必ず運ゲーであるとは断言できません。

この教材では「**固定した初手から、このソルバーが公開情報のみで全安全マスを開ける**」ことを明示的な合格条件にします。初手が変わっても常に同じ条件を満たす、すべての推論規則を網羅している、すべての盤面を等確率で生成する、といった強い条件は主張しません。

参考資料

- Simon Tatham (2019), Writing a soluble-grid generator for Mines：候補の改変とソルバー検査。
- JSMinesweeper：推測不要盤面の生成、ソルバー、分析。
- Mindsweeper：公開されている別の無推測盤面生成の実装。

演習　公開状態と真の地雷配置を別の型で定義してみましょう。また、局所推論しか使わないソルバーが行き詰まるが、集合の差分を使えば先に進める例を考えてみましょう。

Chapter 23

API 索引と次の学習

この章で学ぶこと：使い方が分からない機能を自分で調べ、検証できるようにする。

標準ライブラリは機能名を列挙するだけでは使いこなせません。必要な操作について**引数・戻り値・所有権・失敗・対象ターゲット**を調べる方法を繰り返しましょう。

分類	主な名前	確認すること
数値	Nat, U32, F32, Word	変換と丸め、上限
文字	Char, String	分割、解析、文字境界
列・配列	List, Array	走査、複製、所有権
辞書・集合	Map, Set	文字列キー、検索結果
結果型	Maybe, Result, Either	失敗の情報
入出力	IO, File, Process	ハンドル、終了コード
通信	TCP, UDP, Socket	接続と切断
並行処理	Chan, IO.fork, IO.join	同期とタイムアウト
GUI	Image, App, Window, Event	画面・イベントの環境
音声	Audio	デバイスと書込み
証明	law, Equal, Sigma	仮定と結論、カーネル

公式資料：Bend 2 公式ガイド、公式 Base ライブラリ。

演習　初めて見る `Base` の関数を一つ選び、最小サンプルを作って型検査し、成功時と失敗時の振る舞いを確かめましょう。